Beach Party Bonanza: PRD v1.0

HowdyHack 2026 · 24-hour build · Godot · Team of 4

1. Executive Summary / Problem Statement

Problem: Motion games usually require a headset, console, or special controller, making them hard to try at a party or on campus. Judges also see many hackathon demos they cannot try for themselves.

Solution: A first-person rhythm game that runs on any laptop with a webcam. MediaPipe tracks your hands, so your real flicks become on-screen flicks with no extra hardware. Flick up to knock seagulls out of the air, flick down to push waves back, and keep the vibe meter alive until the song ends.

Value: An instant “aha” demo, zero setup for players, a strong fit for Best Beach Hack, and a clear technical story for judges (real-time hand tracking, latency tuning, beat-synced gameplay).

2. Target Users & Personas

Persona	Context	Key need
Judge (primary for v1)	About 3 minutes at the table, standing, unfamiliar with the game	Understand it in 20 seconds, see it work live, play with no instructions
Casual party player	Laptop at a social gathering, no gaming gear	Fun in under a minute, forgiving timing
Beginner hacker (secondary)	Curious what hand tracking can do	A simple, readable demo

3. In-Scope vs. Out-of-Scope (v1.0)

In scope (must-have):

- Webcam hand tracking with a mirrored corner preview
- Calibration screen
- Up and down flicks for both hands
- One 45-60 second song (already owned) with a hand-authored beatmap
- Vibe meter, win screen, lose screen, replay
- Beach scene, seagulls, waves, hand sprites
- Maroon and white Aggie touches (low cost, helps three prize tracks)

Nice-to-have (only if time allows): 5-second intro zoom, short tutorial card, Reveille cameo, Gig 'em ready gesture

Out of scope for v1:

- **Hold-cup mechanic (planned for generation 4)**
- Drinking mechanic, cup object detection
- Multiple songs, leaderboards, accounts, mobile support
- Auto-generated random obstacle patterns

4. Key Features & Functional Requirements

F1. Hand tracking and gesture events
As a player, I want my real hand movements recognized, so that I can play without a controller.

- Emits `UP` and `DOWN` events per hand (hold detection stays disabled behind a flag)
- One physical flick fires exactly one event (cooldown enforced)
- Left and right hands are correct in a mirrored view
- Tracking runs at 25+ FPS on a standard laptop

F2. Calibration
As a player, I want to set my resting hand position, so that flicks are detected relative to me.

- Prompts the player to rest elbows on a table and hold hands still for 2 seconds
- Stores the baseline; the game starts only after both hands are detected

F3. Obstacle gameplay
As a player, I want seagulls and waves to arrive on the beat, so that it feels like a rhythm game.

- Seagulls are hit by an up flick, waves by a down flick
- Obstacles grow as they approach and are clearly distinguishable from the background
- Hit window of about ±150 ms; outcome is binary (hit or miss)
- Obstacle timing is driven by the audio clock, not frame counting

F4. Vibe meter and win/lose
As a player, I want clear feedback on how I'm doing, so that I know when I'm about to lose.

- Green, yellow, and red bands; each miss costs 10-15%, and streaks regenerate
- Reaching zero shows "Vibe Killed"; surviving to the song's end shows a win screen
- Replay returns to the title screen

F5. Beatmap pipeline
As a developer, I want beats generated once and then hand-edited, so that patterns are fair and fun.

- `librosa` generates candidate timestamps offline from the owned track

- The team edits a JSON beatmap into fixed up/down patterns
- A global latency offset constant can be tuned on the demo laptop

F6. Judge mode
As a judge, I want to see the full loop quickly, so that I understand the game in 90 seconds.

- A 45-60 second track that reaches a win or lose ending

5. User Experience & Flow

1. **Title screen:** press a key to begin
2. **Calibration:** webcam preview in the top right; hands detected, baseline set
3. **Short tutorial card:** up = seagulls, down = waves, how to win and lose
4. **Gameplay:** first-person view, beach party in the side background, obstacles approaching along a path toward the center, hands at the left and right of the screen, vibe meter on the side
5. **End screen:** win or “Vibe Killed”, then replay

Wireframe placeholders: title, calibration, gameplay HUD, end screen.

6. Technical & System Constraints

Engine: Godot (fixed). Godot has no native MediaPipe, so v1 uses a two-part architecture:

Part	Owner	Responsibility
Python tracker (sidecar)	Back-end dev	OpenCV captures frames, MediaPipe Hands finds landmarks, flick detection logic, sends small JSON messages over local UDP (for example `{"hand":"R","event":"UP","t":123.456}` plus a ~30 Hz hand-height stream for the on-screen hands). Never send video.
Godot game	Front-end dev	Listens on a UDP port (`PacketPeerUDP`), runs the beatmap, spawns obstacles, scores hits, vibe meter, UI, scenes

- **Audio sync:** compare obstacle times to the audio playback position, adjusted by mix and output latency (`get_playback_position()` + `AudioServer.get_time_since_last_mix()` - `AudioServer.get_output_latency()`), plus a tunable global offset
- **Performance:** tracking at 25+ FPS, end-to-end input latency under about 100 ms, `model_complexity=0`
- **Mirroring:** flip the frame before MediaPipe so handedness matches the player; test in hour one
- **Packaging risk:** the demo needs Godot plus a running Python process, so write a one-command launcher script and test it on the demo laptop early

- **Audio:** the owned track must be cleared for use in the Devpost demo video
- **Content:** non-alcohol cups (lemonade or sweet tea), no “chug” language
- **Privacy:** webcam video is processed locally and never stored or uploaded
- **Budget:** free tools only

7. Team Roles & Task Split

Role	Owns	First deliverables
Back-end dev	Python tracker, flick detection, UDP messages, librosa beatmap draft	Terminal prints `UP` / `DOWN`, then UDP events Godot can read
Front-end dev	Godot project, obstacle spawning, scoring, vibe meter, screens	Obstacles on the beat with keyboard input as a stand-in, then swap in UDP
Artist / animator	Beach scene, seagulls, waves, hand sprites, UI, maroon and white branding	Placeholder sprites in hour 3, final assets by hour 14
Presenter	Pitch, demo video, Devpost page, GitHub README, backup recording	Pitch script in hour 6, backup video of a full run by hour 20

Keyboard stand-in input lets the front end build and test the whole game before the tracker is ready.

8. Prize Track Strategy

- **Best Beach Hack:** the beach theme is the game itself
- **Best Design (Software):** consistent palette, clean HUD, readable obstacles; be ready to explain color and layout choices
- **Best Aggie-themed:** maroon cups, Reveille on the beach, an A&M-flavored party; a Gig ’em ready gesture if time allows
- **Overall Software:** a rock-solid live demo plus a clear “what was hard” story (latency and audio sync)

9. Metrics & Success Criteria

Metric	Target
Flick detection accuracy (20 flicks each direction)	90%+
Double-fire rate	Under 5%
Tracking FPS on the demo laptop	25+

Time from start to first obstacle for a new player	Under 60 seconds
Consecutive full demo runs with no crash	5 of 5
Self-score on the HowdyHack rubric	85+
Outcome	Placement in Best Beach Hack or Best Design, and a shot at top 3 Overall Software

10. Risks & Open Questions

Risks

- **Gesture reliability:** noisy webcam or bad lighting. *Mitigation:* generous hit window, lenient meter, test in venue lighting.
- **Latency:** camera plus inference delay. *Mitigation:* offset slider, low model complexity.
- **Two-process demo:** Godot plus Python can fail on stage. *Mitigation:* launcher script, early test, backup recording.
- **Scope creep:** intro, tutorial, and extras eat build time. *Mitigation:* freeze the must-have list at hour 2.
- **Rule risk:** all four members must be present and on time; the demo video and GitHub link are required; the project must be built entirely within the 24 hours.

Open questions

1. Which laptop will be the demo machine, and has MediaPipe been tested on it?
2. Is the owned track at a steady tempo, and is it cleared for the demo video?
3. Will the Aggie theme use only original art (no official A&M logos) to avoid branding issues?

11. Roadmap

- **v1 (hackathon):** flicks, one song, vibe meter, win/lose
- **v2:** more songs and patterns, intro animation
- **v3:** polish, difficulty levels, tutorial
- **v4:** hold-cup mechanic

Want me to move on to the Godot UDP listener and beatmap loader, or the hour-by-hour plan?